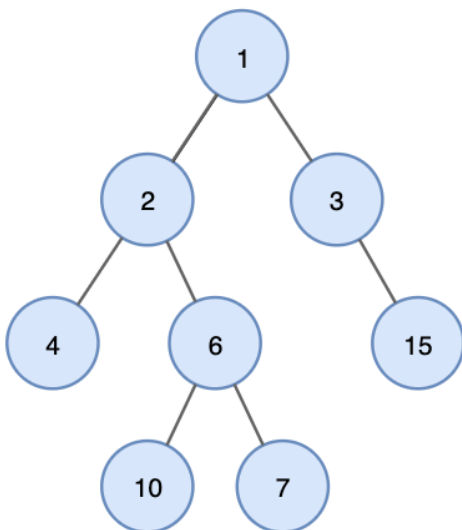


LV16 트리(tree), 그래프(graph) DFS 순회

<https://youtu.be/B3-ETG7aD5k?feature=shared>

트리(Tree) 자료구조

🌳 트리란?



📖 트리(Tree)란 노드들이 나무 가지처럼 연결되어 있는 비선형 계층적 자료구조이다.

- 트리는 다음과 같이 나무를 거꾸로 뒤집어 놓은 모양과 유사하다
- 트리는 또한 트리 내에 다른 하위 트리가 있고 그 하위 트리 안에도 또 다른 하위 트리가 있는 재귀적인 자료구조이기도 합니다

트리의 조건

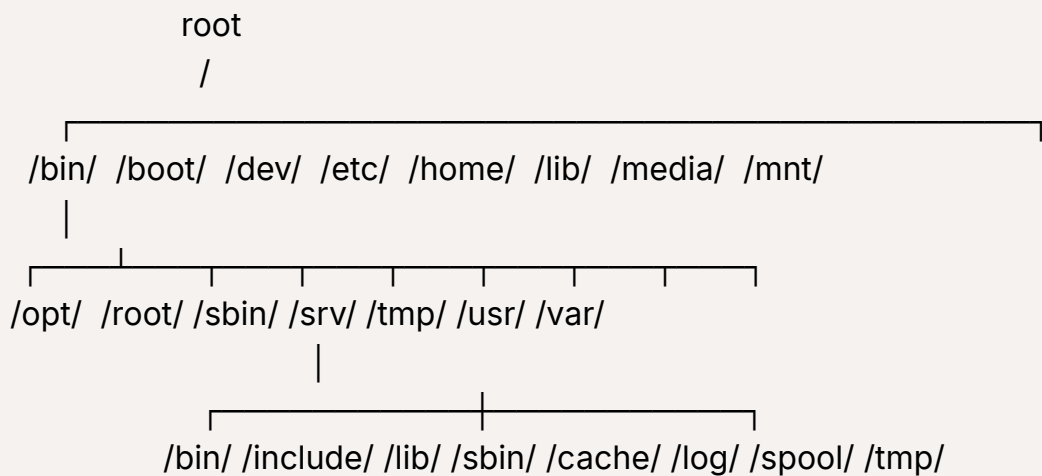
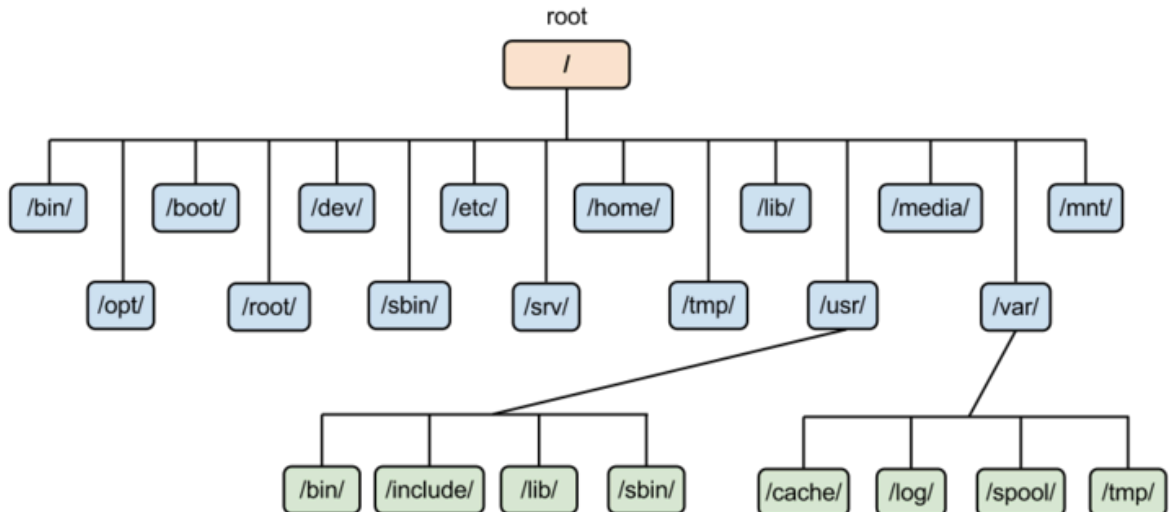
📋 트리의 조건:

1. 단방향 그래프
2. Cycle이 없어야함

3. 부모 자식관계인 그래프(정확히, 두 노드 간의 연결선이 유일한 그래프)

실생활 예시

컴퓨터의 directory구조가 트리 구조의 대표적인 예가 될 수 있습니다.



트리 구조에서 사용되는 기본 용어

기본 개념

- **노드 (Node):** 트리를 구성하고 있는 기본 요소
- 노드에는 키 또는 값과 하위 노드에 대한 포인터를 가지고 있음
- A, B, C, D, E, F, G, H, I, J

노드 관계

- 간선 (Edge): 노드와 노드 간의 연결선
- 루트 노드 (Root Node): 트리 구조에서 부모가 없는 최상위 노드
 - root node : A
- 부모 노드 (Parent Node): 자식 노드를 가진 노드
 - H, I에 부모 노드는 D
- 자식 노드 (Child node): 부모 노드의 하위 노드
 - 노드 D의 자식 노드는 H, I
- 형제 노드 (Sibling node): 같은 부모를 가지는 노드
 - H, I는 같은 부모를 가지는 형제 노드

노드 분류

- 외부 노드(external node, outer node), 단말 노드 (terminal node), 리프 노드 (leaf node)
 - 자식 노드가 없는 노드
 - H, I, J, F, G
- 내부 노드 (internal node, inner node), 비 단말 노드 (non-terminal node), 가지 노드 (branch node)
 - 자식 노드 하나 이상 가진 노드
 - A, B, C, D, E

트리 속성

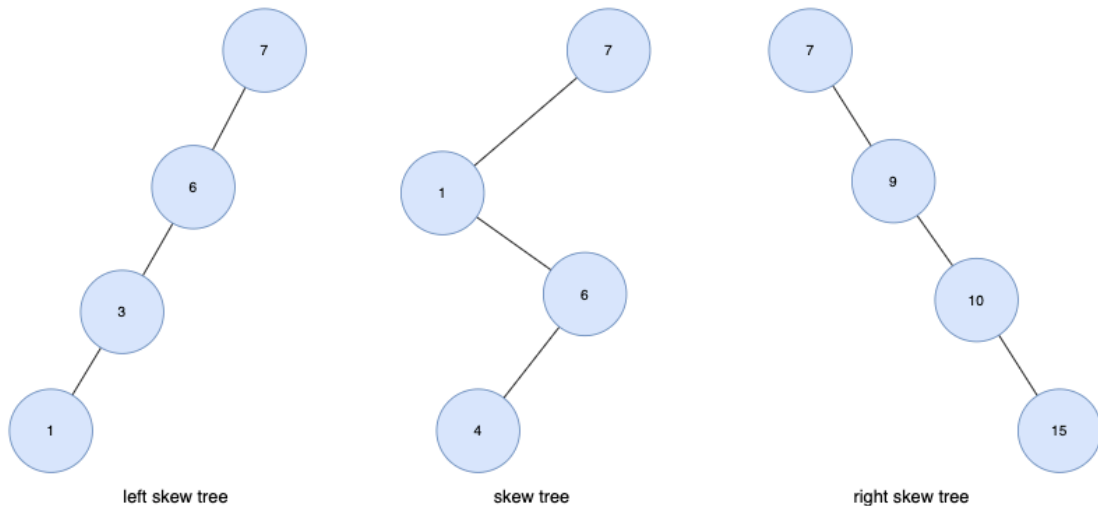
- 깊이 (depth): 루트에서 어떤 노드까지의 간선(Edge) 수
 - 루트 노드의 깊이 : 0
 - D의 깊이 : 2
- 높이 (height): 어떤 노드에서 리프 노드까지 가장 긴 경로의 간선(Edge) 수
 - 리프 노드의 높이 : 0
 - A 노드의 높이 : 3



트리 종류

편향 트리 (Skew Tree)

- 모든 노드들이 자식을 하나만 가진 트리
- 왼쪽 방향으로 자식을 하나씩만 가질 때 left skew tree, 오른쪽 방향으로 하나씩만 가질 때 right skew tree라고 함



이진트리 (Binary Tree)

- 각 노드의 차수(자식 노드)가 2 이하인 트리

이진 탐색 트리 (Binary Search Tree, BST)

- 순서화된 이진 트리
- 노드의 왼쪽 자식은 부모의 값보다 작은 값을 가져야 하며 노드의 오른쪽 자식은 부모의 값보다 큰 값을 가져야 함

m원 탐색 트리(m-way search tree)

- 최대 m개의 서브 트리를 갖는 탐색 트리
- 이진 탐색 트리의 확장된 형태로 높이를 줄이기 위해 사용함

균형 트리 (Balanced Tree, B-Tree)

- m원 탐색 트리에서 높이 균형을 유지하는 트리
- height-balanced m-way tree라고도 함

사용 사례

계층적 데이터 저장

- 트리는 데이터를 계층 구조로 저장하는 데 사용됩니다
- 예를 들어 파일 및 폴더는 계층적 트리 형태로 저장됩니다

효율적인 검색 속도

- 효율적인 삽입, 삭제 및 검색을 위해 트리 구조를 사용합니다

힙(Heap)

- 힙도 트리로 된 자료 구조입니다

데이터 베이스 인덱싱

- 데이터베이스 인덱싱을 구현하는데 트리를 사용합니다
- 예) B-Tree, B+Tree, AVL-Tree..

Trie

- 사전을 저장하는 데 사용되는 특별한 종류의 트리입니다



트리 하드코딩 하기

1 인접행렬로 표현하는 경우

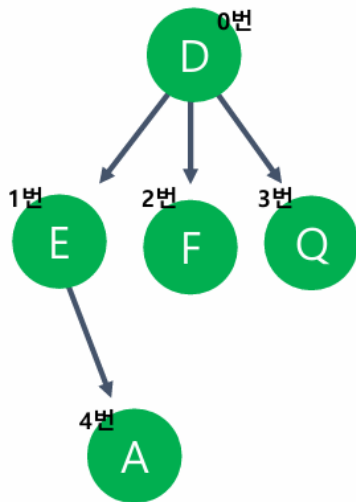


트리 구조:

```
      D(0번)
     /  |  \
    E(1) F(2) Q(3)
   /
  A(4)
```



인접행렬 표현:

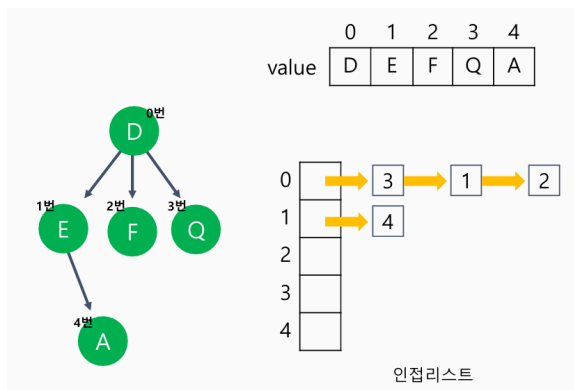


```

char value[5] = { 'D', 'E', 'F', 'Q', 'A' };
int matrix[5][5] =
{
    0,1,1,1,0, // D → E, F, Q 연결
    0,0,0,0,1, // E → A 연결
    0,0,0,0,0, // F 연결 없음
    0,0,0,0,0, // Q 연결 없음
    0,0,0,0,0, // A 연결 없음
};
  
```

2 리스트로 표현하는 경우

🔗 인접리스트:



0: → 3 → 1 → 2

1: → 4

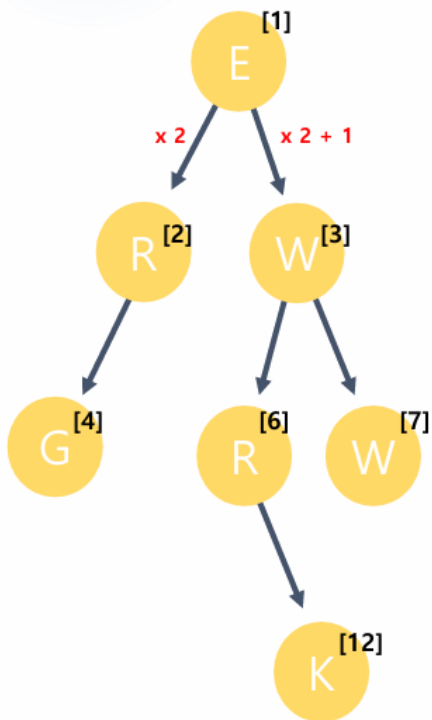
2:

3:

4:

3 1차원 배열로 표현하는 경우

📄 2진트리의 경우 1차원배열로도 표현 가능하다:



```
char map[256] = " ERWG RW  K
";
```

// 배열 인덱스와 트리 노드 관계:

// E[1]

// / \

// R[2] W[3]

// / \ / \

// G[4] [5]R[6] W[7]

// \

// K[12]

// rootNode를 1번으로

// 왼쪽자식은 부모 index * 2

// 오른쪽 자식은 부모 index * 2 + 1

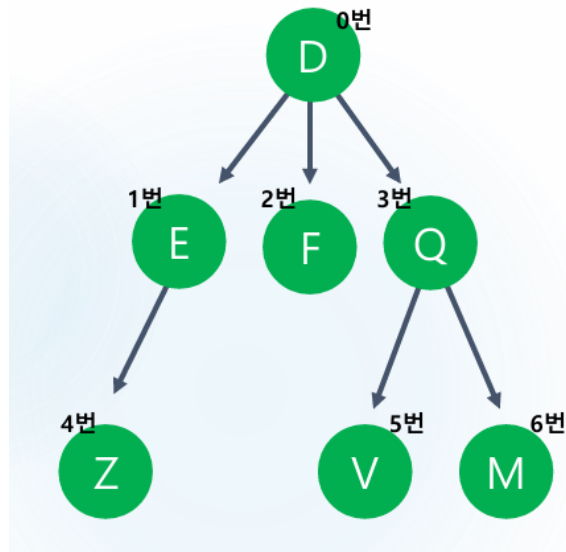
```
char map[256] = " ERWG RW  K";
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
map		E	R	W	G		R	W					K	

💡 인덱스 계산 공식:

- 왼쪽 자식: $\text{parent_index} * 2$
- 오른쪽 자식: $\text{parent_index} * 2 + 1$
- 부모: $\text{child_index} / 2$

🔄 자식노드 출력해보기



```

#include<iostream>

int main()
{
    char value[10] = "DEFQZVN";
    int map[7][7] =
    {
        0,1,1,1,0,0,0,
        0,0,0,0,1,0,0,
        0,0,0,0,0,0,0,
        0,0,0,0,0,1,1,
        0,0,0,0,0,0,0,
        0,0,0,0,0,0,0,
        0,0,0,0,0,0,0,
    };

    int n = 0;
    std::cin >> n;

    for (size_t i = 0; i < 7; i++)
    {
        if (map[n][i] > 0)
        {
            std::cout << value[i] << std::endl;
        }
    }
}

```



```

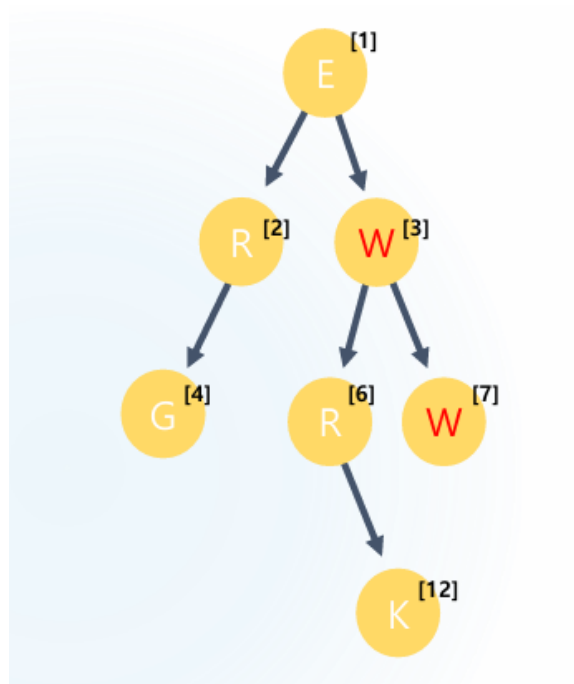
    }
}

return 0;
}

```

☁️ **동작:** 입력받은 노드 번호에 자식노드들을 전부 출력해보자.

🔍 부모 노드 출력 해보기



```

#include<iostream>

int main()
{
    char map[15] = " ERMG RW  K ";

    int n = 0;
    std::cin >> n;

    std::cout << map[n / 2] << std::endl;
}

```

```
return 0;  
}
```

☁️ 동작: 1차원 배열로 구현된 트리에서 부모 노드 찾기

🎯 트리 순회와 DFS 연결

그래프 순회 (DFS)

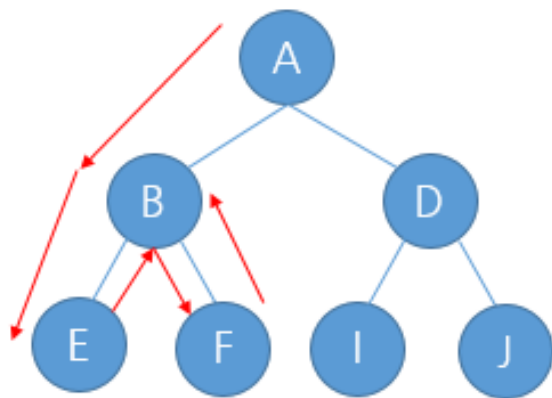
DFS

재귀호출, 스택을 사용하여 노드를 탐색한다. Depth First Search(깊이 우선 탐색)
노드를 깊숙히 안쪽으로 들어가면서 탐색한다.
트리, 그래프 둘다 연습해야한다.

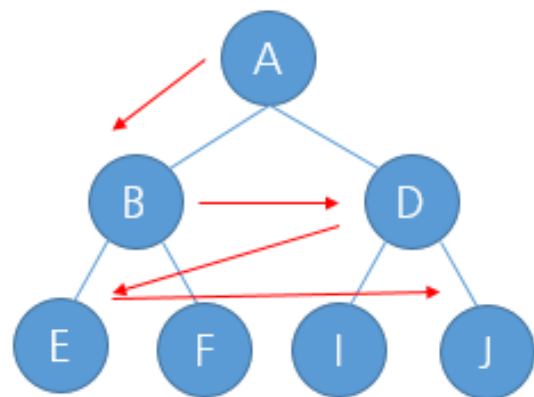
BFS

큐를 이용하여, 노드를 탐색한다. Breadth First Search(너비 우선 탐색)
노드의 레벨 또는 너비를 우선적으로 탐색한다.
트리, 그래프 둘다 연습해야한다.

트리나 그래프냐에 따라서 큰 순회방법은 비슷하지만
특정조건이 들어갈수 있기때문에 구현방법이 달라질수 있다.



DFS
A B E F D I J 순서로 방문



BFS
A B D E F I J 순서로 방문

📊 핵심 정리

트리 vs 그래프 차이점

구분	트리	그래프
사이클	없음	있을 수 있음
루트	하나의 루트 존재	루트 개념 없음
방향성	부모→자식 (단방향)	양방향/단방향 가능
연결성	모든 노드 연결	연결되지 않은 노드 가능

표현 방법 비교

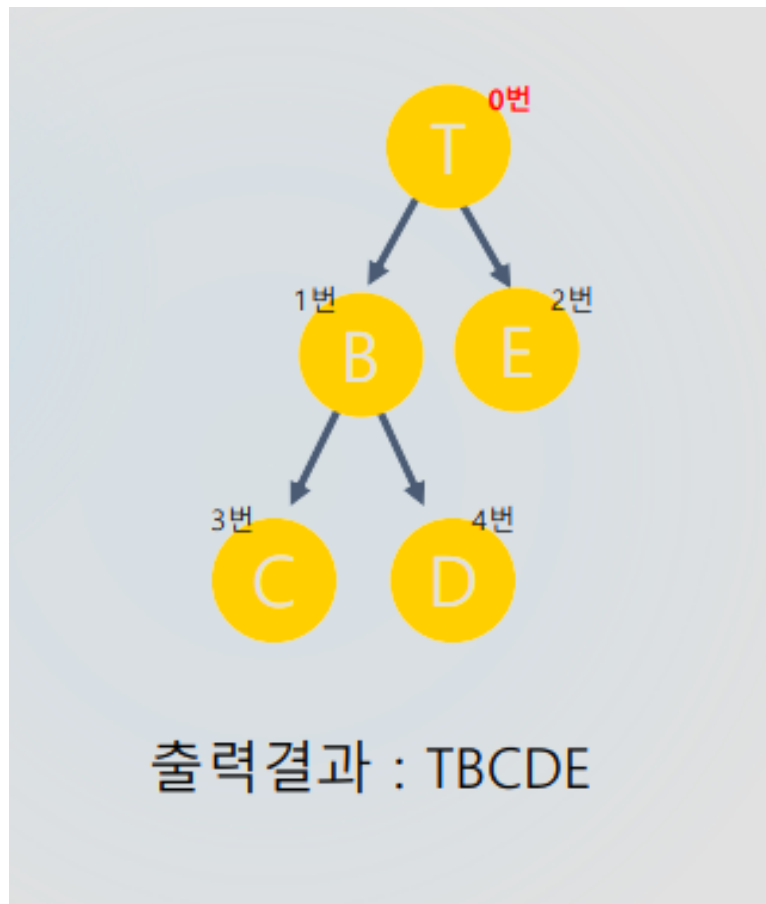
방법	장점	단점	적합한 경우
인접행렬	구현 간단, 빠른 연결 확인	메모리 낭비	작은 트리, 완전 트리
인접리스트	메모리 효율적	구현 복잡	큰 트리, 희소 트리
1차원 배열	매우 간단	완전 이진트리만 가능	힙, 완전 이진트리

활용 분야

- 📁 파일 시스템: 디렉토리 구조
- 🔍 검색 엔진: 인덱싱 구조
- 💾 데이터베이스: B-Tree 인덱스
- 💬 언어 처리: 구문 분석 트리
- 🎮 게임: 의사결정 트리

트리는 계층적 구조를 효율적으로 표현하고 관리할 수 있는 핵심적인 자료구조입니다! 🚀

1. 트리 순회 (Tree Traversal)



기본 개념

- **트리**: 사이클이 없는 연결 그래프
- **DFS**: 한 방향으로 끝까지 가서 탐색 후 되돌아가기

코드 구조

```
#include <iostream>

char value[6] = "TBECD";
char path[6] = "";

int map[5][5] =
{
    0,1,1,0,0,
    0,0,0,1,1,
    0,0,0,0,0,
    0,0,0,0,0,
    0,0,0,0,0,
```

```

};

void run(int now, int level)
{
    std::cout << value[now];

    for (size_t i = 0; i < 5; i++)
    {
        if (map[now][i] == 1)
        {
            path[level + 1] = value[i];
            run(i, level + 1);
            path[level + 1] = 0;
        }
    }
}

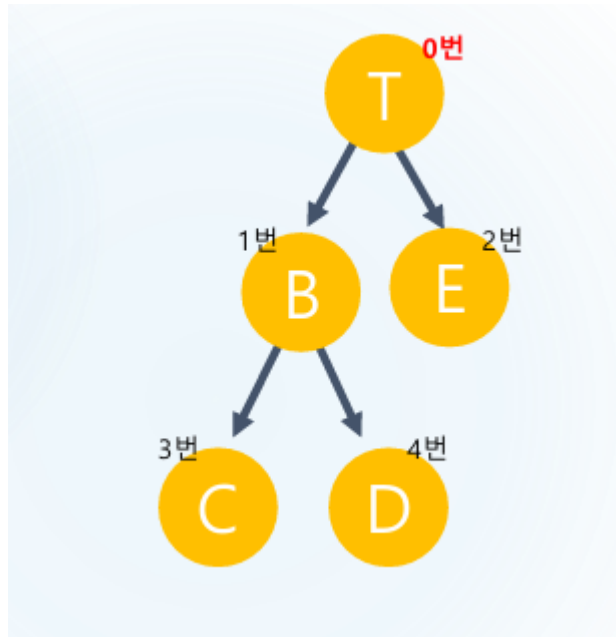
int main()
{
    path[0] = value[0];
    run(0, 0);

    return 0;
}

```

실행 결과: T → B → C → D → E (TBCDE)

2. 배열 기반 트리 표현



- 1차원 배열로 이진트리 표현
- 인덱스 규칙: 왼쪽 자식($\text{now} * 2$), 오른쪽 자식($\text{now} * 2 + 1$)

코드

```

#include <iostream>

char tree[256] = " TBECD";

void run(int now, int level)
{
    if (tree[now] == '\0')
    {
        return;
    }

    std::cout << tree[now];
    run(now * 2, level + 1);
    run(now * 2 + 1, level + 1);
}

int main()
{

```

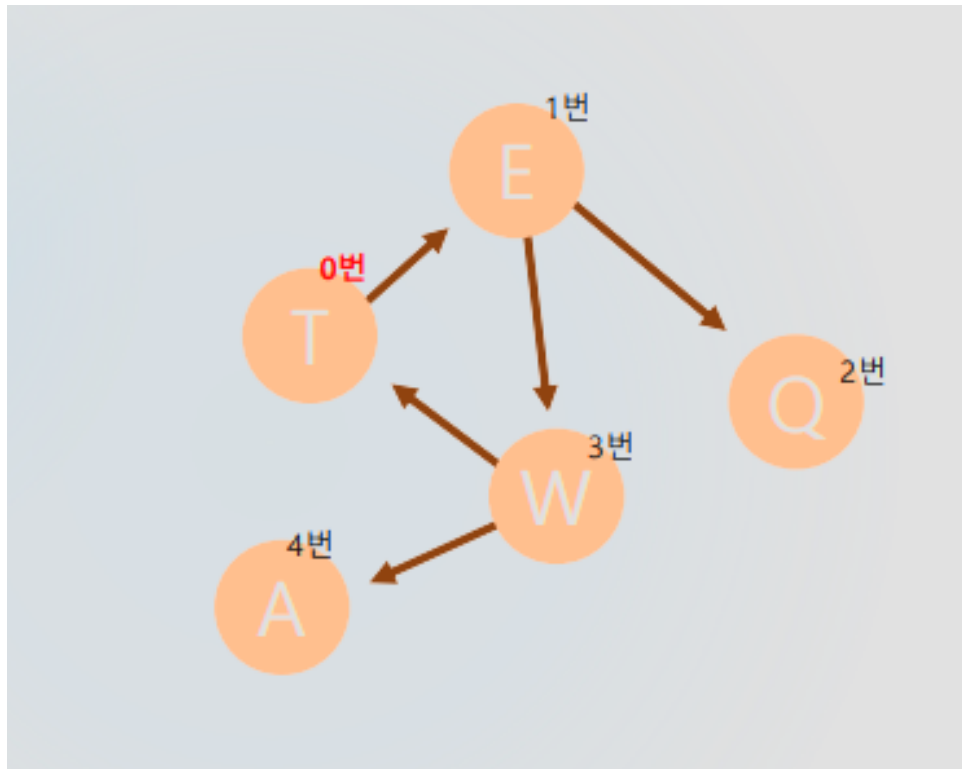
```

run(1, 0);

return 0;
}

```

3. 그래프 순회 (Graph Traversal)



핵심 차이점

- 사이클 존재 가능 → `visited[]` 배열 필수
- 중복 방문 방지가 핵심

코드 구조

```

#include <iostream>

char value[10] = "TEQWA";
int map[5][5] =
{
    {0, 1, 0, 0, 0},

```

```

    {0, 0, 1, 1, 0},
    {0, 0, 0, 0, 0},
    {1, 0, 0, 0, 1},
    {0, 0, 0, 0, 0}
};

int visited[10] = { 0, }; // 방문 표시
char path[10] = { 0, }; // 경로 저장

void run(int now, int level)
{
    std::cout << value[now];

    for (size_t i = 0; i < 5; i++)
    {
        if (map[now][i] == 1 && visited[i] == 0) // 연결되고 미방문
        {
            path[level + 1] = value[i];
            visited[i] = 1; // ★ 방문 표시
            run(i, level + 1);
            path[level + 1] = 0;
        }
    }
}

int main()
{
    path[0] = 'T';
    visited[0] = 1; // 시작점 방문 표시
    run(0, 0);

    return 0;
}

```

그래프 예시

- 노드: T, I, Q, W, A
- 연결 관계: T가 루트, 각 노드들이 상호 연결

- 탐색 순서: visited 배열에 따라 결정

4. 핵심 비교

구분	트리 DFS	그래프 DFS
사이클	없음	있을 수 있음
visited 배열	불필요	필수
복잡도	간단	중복 체크 필요

5. 중요 포인트

반드시 기억할 것

1. 재귀 구조: 작은 문제로 나눠서 해결
2. **visited 배열**: 그래프에서 무한루프 방지의 핵심
3. 초기화: `visited[시작점] = 1` 잊지 말기

코딩 주의사항

- 배열 범위 체크
- 재귀 종료조건 명확히
- visited 배열 올바른 사용

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 '**연습**'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얀얀코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얀얀코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얀얀코딩 상담방](#)

▼ oopy:hide



문제 1번

현수는 다른 조직으로 이직에 성공했습니다.

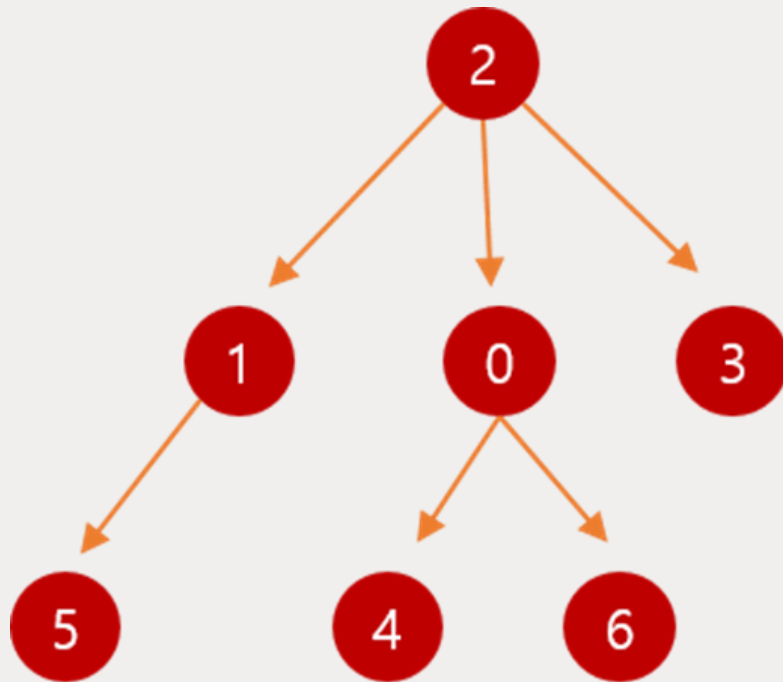
이 그룹의 조직도를 인접행렬($N \times N$ 사이즈)로 전달 받으면, **현수의 직속 보스와 직속 부하들이 누군지 출력** 해 주세요.

문제 조건

1. 현수는 0번 노드입니다.
2. 부하들끼리 번호 순서대로 출력 해 주세요

예시

만약 아래와 같은 인접행렬을 입력받았다면,



boss:2

under:4 6

입력 예제

7

0 0 0 0 1 0 1

0 0 0 0 0 1 0

1 1 0 1 0 0 0

0 0 0 0 0 0 0

0 0 0 0 0 0 0

0 0 0 0 0 0 0

0 0 0 0 0 0 0

출력 결과

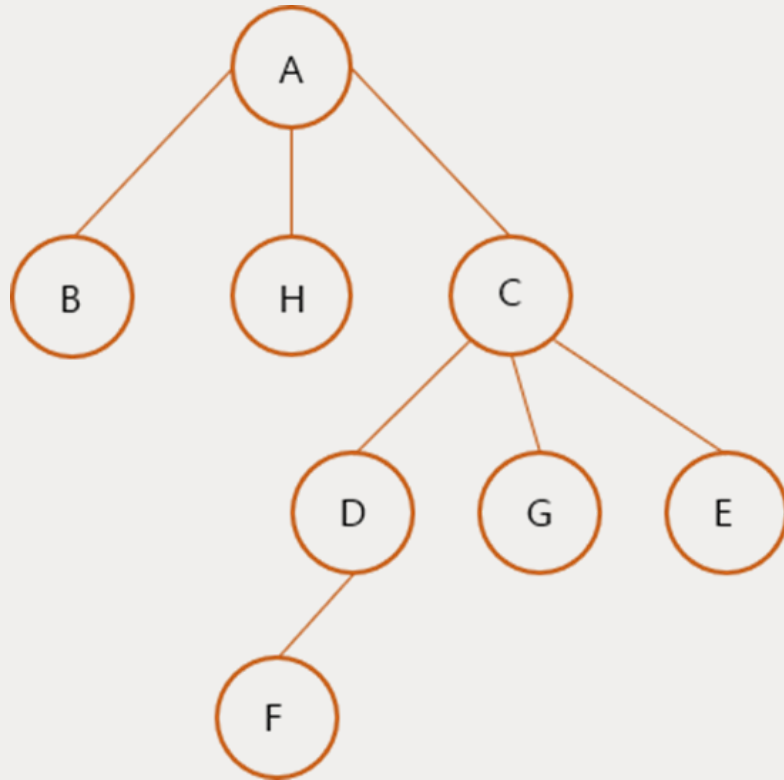
boss:2

under:4 6



문제 2번

마마코코 가족의 계보는 다음과 같습니다. (A ~ H)



다음과 같은 그래프를 **인접행렬 (2차배열)**로 하드코딩 하주세요.

이제 **노드이름을 입력받고, 그 노드의 형제들을 모두 출력** 하주세요.

(만약 형제들이 없다면 "**없음**" 을 출력해주시면 됩니다.)

예를들어 H의 형제는 B와 C입니다.

A의 형제는 없습니다.

C의 형제는 B와 H입니다.

입력 예제

H

출력 결과

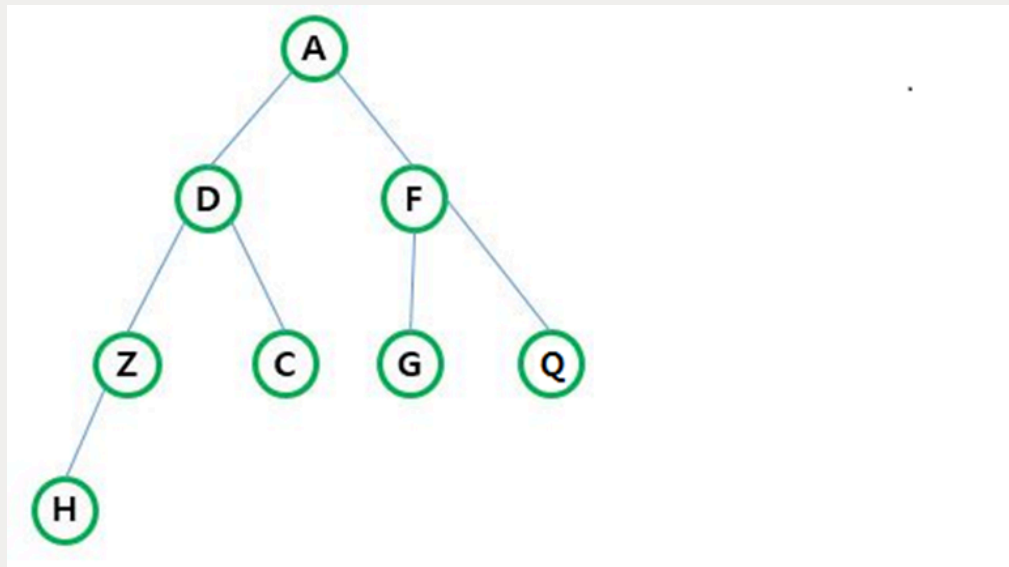
B C



문제 3번

아래 이진 트리를 **1차원 배열**에 저장하세요.

(Root 노드인 'A'를 1번 Index에 두는 것을 잊지마세요.)



이제 문자 2개를 입력 받으세요.

그 문자에 해당하는 노드가 서로 부모자식 관계인지 아닌지 출력 하세요.

ex)

G F → 부모자식관계

ex)

Z C → 아님

입력 예제

G F

출력 결과

부모자식관계



문제 4번

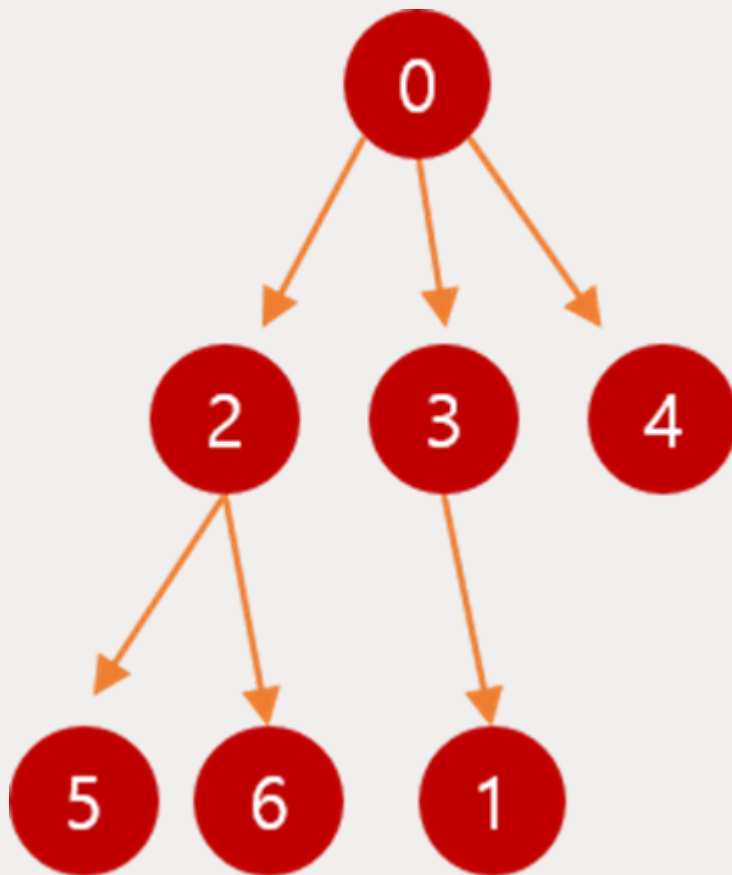
DFS는 그래프 / 트리를 탐색하는 방법입니다. 배열 or 링크드리스트 자료구조는 for문으로 쉽게 탐색이 가능하지만,

그래프 같은 자료구조는 for문으로 탐색이 어렵습니다. DFS로 탐색을 하곤 합니다.

N x N 그래프를 인접행렬로 입력받고,

DFS 탐색 순서대로 출력 해 주세요.

1. ex) 만약 아래와 같은 인접행렬을 입력받았다면,



출력결과 : 0 2 5 6 3 1 4

입력 예제

7

0 0 1 1 1 0 0

0 0 0 0 0 0 0

0 0 0 0 0 1 1

0 1 0 0 0 0 0

0 0 0 0 0 0 0

0 0 0 0 0 0 0

0 0 0 0 0 0 0

출력 결과

0 2 5 6 3 1 4

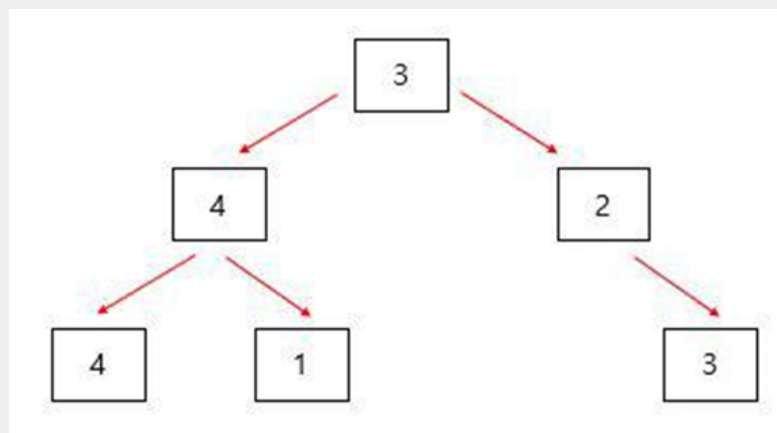


문제 5번

다음은 하드코딩 해주세요.

0	1	2	3	4	5	6	7
0	3	4	2	4	1	0	3

위 1차배열은 다음 이진 트리를 나타냅니다.



이제 변경할 index와 값을 입력받아주세요.

만약 4 7 을 입력 받았다면,

배열의 4번 index의 값을 7로 바꾼 후 DFS를 돌리면 됩니다.

DFS 돌린 결과를 출력 해 주세요.

입력 예제

4 7

출력 결과

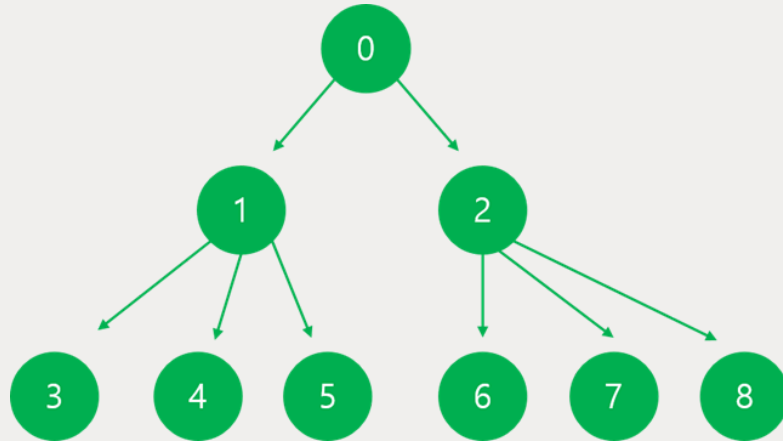
3 4 7 1 2 3



문제 6번

N x N인접행렬 트리를 입력받아주세요.

Level 2에 도착했을 때 마다 경로를 출력하며 됩니다.



만약 위와 같은 트리를 입력받았다면, 다음과 같이 출력하면 됩니다.

0 1 3

0 1 4

0 1 5

0 2 6

0 2 7

0 2 8

입력 예제

9

0 1 1 0 0 0 0 0 0

0 0 0 1 1 1 0 0 0

0 0 0 0 0 0 1 1 1

0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0

출력 결과

0 1 3

0 1 4

0 1 5

0 2 6

0 2 7

0 2 8



문제 7번

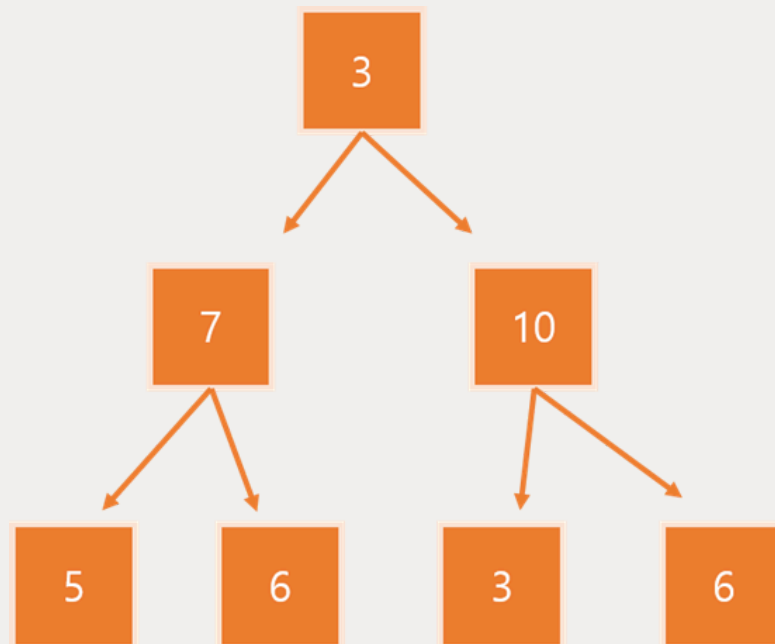
최대 Level이 2인, Binary Tree를 1차원 배열에 입력받아주세요.

(숫자 8개 입력, 숫자 0 은 없는 노드입니다.)

짝수 노드를 발견할 때마다

탐색을 멈추고, 왔었던 경로를 출력하시면 됩니다.

만약 아래 트리와 같이 입력받았다면



입력 예제

0 3 7 10 5 6 3 6

출력 결과

3 7 6

3 10

3 10 6



문제 1번

문자와 숫자로 구성된 세트를 n개 입력 받습니다.

(노드를 만들고 구조체 배열에 입력 받아주세요)

아래와 같은 우선순위를 기준으로, 정렬 후 결과를 출력 해주세요.

- 정렬 우선순위

1. 문자 우선
2. 같은 문자라면 숫자를 기준으로 정렬



입력 예제

8

A 3

C 2

H 9

I 8

H 2

C 9

C 1

A 10

출력 결과

A 3

A 10

C 1

C 2

C 9

H 2

H 9

I 8



문제 2번

Level depth와 **Branch수**를 입력 받으세요.

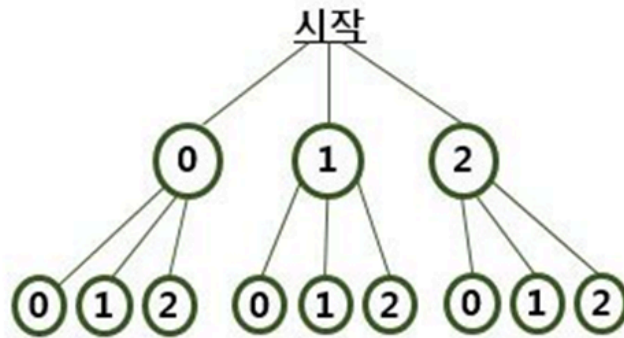
아래 그림과 같이 동작하는 재귀호출을 구현하고,

마지막 Level 에서 경로를 **모두 출력** 해주세요.

Ex)

Level depth = 2 ↙

자식노드 = 3 ↙



<출력결과>

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

입력 예제

2

3

출력 결과

0 0

0 1

0 2

1 0

1 1

1 2

2 0

2 1

2 2



문제 3번

6칸짜리 두 배열이 있습니다.

두 배열에 6자리 숫자를 각각 채워주세요.

그리고 **두 숫자의 합을 출력** 해주세요.

(두 수의 합이 1,000,000보다 작은 입력값으로 주어집니다.)

자릿수 올림 처리를 잊지마세요

ex)

351429

+ 137735

489164

입력 예제

3 5 1 4 2 9

1 3 7 7 3 5

출력 결과

4 8 9 1 6 4



문제 4번

5개의 숫자를 배열에 입력 받으세요.

이 숫자가 증가되는 숫자로 되어 있는지 확인하는 프로그램을 작성해주세요.

(Hint: 1중 for문을 사용하면 됩니다)

증가되는 숫자로 되어 있으면 "증가됨"

아니면 "증가안됨" 이라고 출력 해주세요.

ex)

입력: 3 9 7 10 5

3	9	7	10	5
---	---	---	----	---

출력: 증가안됨

입력 예제

3 9 7 10 5

출력 결과

증가안됨



문제 5번

다섯 자리 숫자 1개를 입력으세요. (10000 ~ 89999 사이 숫자)

그리고 이 숫자를 분해하여 배열에 각 숫자들을 집어 넣어 주세요.

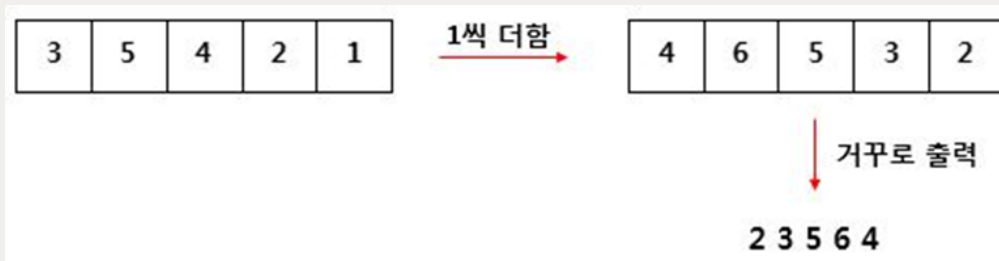
그 다음 각각 1씩 더하여 거꾸로 출력 하면 됩니다.

(자리올림을 신경쓰셔야 합니다)

ex)

입력: 35421

//띄어쓰기 없이 숫자 1개를 입력 받습니다



출력: 23564

//띄어쓰기 없이 출력하시면 됩니다

[힌트]

10 과 %10을 반복하면 숫자를 하나씩 꺼낼 수 있어요

입력 예제

35421

출력 결과

23564



문제 6번

한 문장을 입력 받으세요.(최대 10글자)

도플이니셜(같은 알파벳이 있는 문자)들을 찾고, 정렬해서 출력 해주세요.

만약 ATKPGTBA를 입력받았다면,

A와 T만 2개 이상 있으므로, 도플이니셜입니다.

ex)



입력 예제

ATKPGTBA

출력 결과

AT



문제 7번

2차배열에 1~6까지의 숫자가 있습니다.

아래 **2차배열을 하드코딩** 해주세요.

그리고 숫자들의 빈도수를 그래프로 표현해주세요.(**Direct Addressing Table 이용**)

빈도수 그래프를 화면에 그대로 표현(출력) 해주시면 됩니다.

3	5	1
3	1	2
3	4	6
5	4	6

출력결과

```
1 * *
2 *
3 * * *
4 * *
5 * *
6 * *
```

출력 결과

1 * *

2 *

3 * * *

4 * *

5 * *

6 * *

